

Catatan Praktikum Minggu 2

Mata Kuliah: Pemrograman Web

Nama: Nova Reskianti

NIM: 10241058

READ: Telusuri satu request penuh

Ambil route /tentang yang Anda buat minggu lalu. Tanpa AI, tulis di catatan Anda:

1. Baris mana di `routes/web.php` yang menangkapnya?

Baris yang menangkap route /tentang adalah baris ke 10 :

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\CourseController;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/tentang', function () {
    return view('tentang');
});
```

2. Kalau ditangani controller, berkas dan method mana?

Tidak ada controller, karena route `/tentang` masih menggunakan closure dan langsung mengembalikan view tentang. Route tersebut tidak menunjuk ke controller seperti `[NamaController::class, 'method']`, tetapi langsung menjalankan fungsi `function ()`.

3. View mana yang dikembalikan? Di path apa persisnya?

View yang dikembalikan adalah `tentang`, yang ditunjukkan oleh `return view('tentang')` pada `routes/web.php`.

```
Route::get('/tentang', function () {
    return view('tentang');
```

File view tersebut berada di `resources/views/tentang.blade.php`

4. Layout apa yang membungkusnya?

Halaman `/tentang` saat ini belum menggunakan layout atau komponen Blade. Hal ini dapat dilihat dari isi `resources/views/tentang.blade.php` yang langsung berisi struktur lengkap HTML seperti `<!DOCTYPE html>`, `<head>`, `<body>`, CSS, dan seluruh konten halaman. Selain itu, tidak terdapat penggunaan `<x-layout>` maupun `@extends` yang menunjukkan bahwa halaman tersebut menggunakan layout. Oleh karena itu, seluruh struktur halaman saat ini ditulis langsung di file `tentang.blade.php`.

5. Jalankan php artisan route:list --path=tentang. Cocok dengan analisis Anda?

Berdasarkan kode pada `routes/web.php`, saya memperkirakan route `/tentang` menggunakan method `GET` dan didefinisikan pada baris 10. Karena menggunakan `Route::get()`, pada `route:list` kemungkinan akan ditampilkan sebagai `GET|HEAD`.

Hasil `php artisan route:list --path=tentang` :

```
PS D:\Nova R\Kuliah\Semeter 5\Proweb\Laravel\kampuslms> herd php artisan
route:list --path=tentang

GET|HEAD      tentang
.....
..... routes/web.php:10

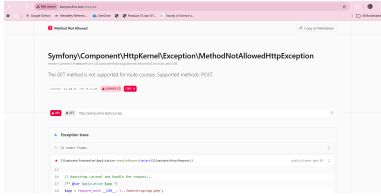
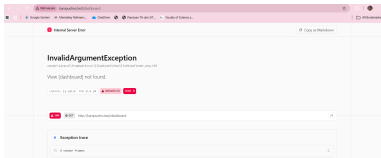
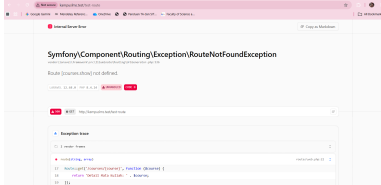
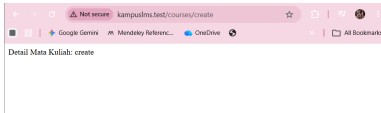
Showing [1] routes
```

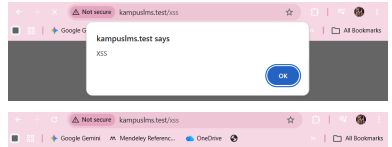
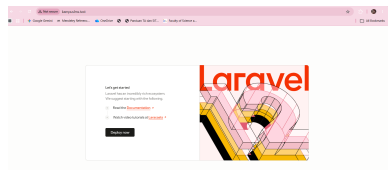
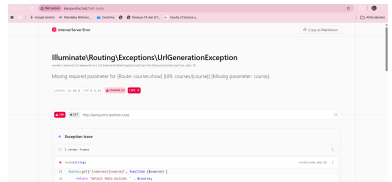
Kesimpulan :

Analisa saya cocok dengan hasil `herd php artisan route:list --path=tentang`. Route `/tentang` menggunakan `GET|HEAD` dan berada di `routes/web.php` pada baris 10.

BREAK: Delapan kerusakan

#	Yang dirusak	Yang Anda pelajari	Prediksi	Hasil Pengujian
---	--------------	--------------------	----------	-----------------

#	Yang dirusak	Yang Anda pelajari	Prediksi	Hasil Pengujian
1	Ubah <code>Route::get</code> menjadi <code>Route::post</code> pada route daftar mata kuliah	Method HTTP tidak cocok → 405	Halaman tidak akan bisa dibuka karena browser mengirim <code>GET</code> , namun route meminta <code>POST</code> .	Muncul error 405 Method Not Allowed dengan pesan: <i>"The GET method is not supported for route courses. Supported methods: POST."</i> 
2	Ubah nama view di <code>return view(...)</code> menjadi yang tidak ada	Exception view not found	Akan terjadi error karena file view yang dipanggil tidak ditemukan.	Muncul error View [dashboard] not found. karena file <code>dashboard.blade.php</code> tidak ditemukan. 
3	Hapus - <code>>name('courses.show')</code> , lalu muat halaman yang memakai <code>route('courses.show')</code>	Kenapa nama route wajib	Akan terjadi error karena route bernama <code>courses.show</code> sudah tidak terdaftar.	Muncul Route [courses.show] not defined. (RouteNotFoundException). 
4	Pindahkan <code>/courses/{course}</code> ke ATAS <code>/courses/create</code> , lalu buka <code>/courses/create</code>	Urutan route menentukan	<code>create</code> dianggap sebagai nilai <code>{course}</code>	Muncul Detail Mata Kuliah: create , sehingga terbukti urutan route menentukan pencocokan route. 

#	Yang dirusak	Yang Anda pelajari	Prediksi	Hasil Pengujian
5	Ganti <code>{{ \$nama }}</code> menjadi <code>{!! \$nama !!}</code> , isi <code>\$nama</code> dengan <code><script>alert('XSS')</script></code>	XSS nyata di layar Anda sendiri	JavaScript akan dieksekusi karena output tidak di-escape.	Muncul popup XSS pada browser.  Uji XSS Nama:
6	Hapus <code>@vite(...)</code> dari layout	Aset tidak termuat	Aset CSS/JS dari Vite tidak akan dimuat sehingga tampilan halaman dapat berubah atau menjadi tidak ter-styling.	
7	Hentikan <code>npm run dev</code> lalu muat ulang halaman	Beda dev server vs build	Development server Vite seharusnya berhenti sehingga aset yang bergantung pada Vite tidak dapat dimuat/diperbarui.	<code>npm run dev</code> tidak dapat dijalankan karena Vite tidak dikenali ('vite' is not recognized).
8	Panggil <code>route('courses.show')</code> tanpa mengirim parameter	Missing required parameter	Akan terjadi error karena <code>route courses.show</code> membutuhkan parameter <code>{course}</code> .	Muncul Missing required parameter ... [Missing parameter: course] dengan UrlGenerationException . 

FIX: Perbaikan Proyek Cacat (Branch **w02**)

Pada branch **w02** di repositori latihan **kampus1ms-broken**, ditemukan **6 masalah** yang perlu dianalisis dan diperbaiki sebagai berikut:

#	Masalah	Lokasi Berkas	Analisis Risiko	Solusi Perbaikan
---	---------	---------------	-----------------	------------------

#	Masalah	Lokasi Berkas	Analisis Risiko	Solusi Perbaikan
1	Urutan route tidak tepat sehingga saling menutupi	<code>routes/web.php</code>	Route dengan parameter wildcard dapat menangkap path yang seharusnya digunakan oleh route statis. Akibatnya, halaman seperti <code>/courses/create</code> tidak dapat diakses dengan benar.	Letakkan route statis seperti <code>/courses/create</code> sebelum route yang menggunakan parameter seperti <code>/courses/{course}</code> .
2	Penggunaan method HTTP yang tidak sesuai, misalnya aksi hapus menggunakan <code>GET</code>	<code>routes/web.php</code>	Route penghapusan yang menggunakan <code>GET</code> dapat terpanggil secara tidak sengaja, misalnya melalui prefetching browser atau crawler.	Gunakan method <code>DELETE</code> atau <code>POST</code> untuk proses penghapusan dan panggil melalui form yang dilengkapi token <code>@csrf</code> .
3	URL ditulis secara langsung (<i>hardcode</i>) pada tautan	Blade view	Jika struktur atau prefix URL berubah, tautan yang ditulis secara langsung dapat menyebabkan <i>broken link</i> .	Ganti URL <i>hardcode</i> dengan helper <code>route('nama.route')</code> menggunakan named route.
4	URL <i>hardcode</i> digunakan pada navigasi dan tombol aksi	Blade view	Penggunaan URL secara langsung membuat pengelolaan route menjadi kurang fleksibel karena nama dan struktur URL tidak terpusat.	Gunakan named route pada seluruh hyperlink, navigasi, dan tombol yang mengarah ke halaman tertentu.
5	Penggunaan sintaks raw HTML <code>{!! !!}</code> tanpa proses sanitasi	Blade view	Data yang ditampilkan menggunakan raw HTML dapat membuka celah Cross-Site Scripting (XSS), terutama jika data berasal dari input pengguna.	Gunakan sintaks <code>{{ }}</code> agar data otomatis di- <i>escape</i> dan lebih aman dari penyisipan kode berbahaya.
6	Query atau logika bisnis ditempatkan langsung di dalam View	Blade view	Hal ini tidak sesuai dengan prinsip MVC karena View menjadi lebih berat, sulit diuji (<i>untestable</i>), serta mencampurkan bagian tampilan dengan data layer.	Pindahkan seluruh proses pengambilan data dan logika bisnis ke Controller, kemudian kirimkan hasilnya ke View.

Catatan: Jika repositori `kampuslms-broken` belum dipublikasikan oleh pengampu, analisis terhadap keenam masalah dan risiko di atas dapat didokumentasikan sebagai pemenuhan modul.

BUILD — Kerangka KampusLMS

Pada tahap BUILD Minggu 2, kelompok kami membangun kerangka dasar aplikasi KampusLMS yang mencakup sistem *layouting*, rute, kontrol logika data statis, dan antarmuka pengguna berbasis Blade.

1. Pembagian Tugas Commit Kelompok 05

Untuk memastikan kolaborasi Git berjalan sesuai prinsip *Continuous Integration* dan setiap anggota memiliki kontribusi nyata yang terisolasi serta dapat direview, tim menyepakati pembagian tugas commit fitur BUILD sebagai berikut:

No	Komponen Fitur BUILD	Penanggung Jawab	Deskripsi Tanggung Jawab
1	Layout (components/layout.blade.php)	Muhammad Rifa Al Rizqul Aulia	Merancang master layout komponen Blade <x-layout> , navbar terpusat, integrasi @vite , slot konten, dan styling semantik.
2	Controller (CourseController.php)	Nova Reskianti (<i>Saya</i>)	Membangun method index() dan show() dengan data statis array mata kuliah.
3	View Index (courses/index.blade.php)	Muhammad Farin Murtadho Syafiq	Membuat tabel responsif penyajian daftar seluruh mata kuliah.
4	View Detail (courses/show.blade.php)	Muhammad Yuspa Ardiansyah	Merancang tampilan informasi lengkap spesifik satu mata kuliah.
5	View Error 404 (errors/404.blade.php)	Muhammad Zaldy Syah Firaz	Mengembangkan halaman fallback penanganan rute atau entitas yang tidak ditemukan.

2. Implementasi Bagian Saya : Controller

Pada tahap BUILD Minggu 2, saya bertanggung jawab mengembangkan bagian **Controller**, yaitu:

app/Http/Controllers/CourseController.php

CourseController digunakan untuk menangani proses yang berkaitan dengan data mata kuliah. Pada implementasi terbaru, controller sudah menggunakan database dan menangani proses CRUD.

a. Method yang digunakan yaitu:

Method	Fungsi
index()	Menampilkan daftar mata kuliah
create()	Menampilkan form tambah mata kuliah

Method	Fungsi
<code>store()</code>	Menyimpan mata kuliah baru
<code>show()</code>	Menampilkan detail mata kuliah
<code>edit()</code>	Menampilkan form edit mata kuliah
<code>update()</code>	Memperbarui data mata kuliah
<code>destroy()</code>	Menghapus mata kuliah

3. Implementasi CourseController

```
<?php

namespace App\Http\Controllers;

use App\Models\Course;
use App\Models\User;
use Illuminate\Http\Request;

class CourseController extends Controller
{
    // Menampilkan semua mata kuliah
    public function index()
    {
        $courses = Course::with('lecturer')->latest()->get();

        return view('courses.index', compact('courses'));
    }

    // Menampilkan form tambah mata kuliah
    public function create()
    {
        $lecturers = User::where('role', 'dosen')->get();

        return view('courses.create', compact('lecturers'));
    }

    // Menyimpan mata kuliah baru
    public function store(Request $request)
    {
        $data = $request->validate([
            'code' => ['required', 'string', 'unique:courses,code'],
            'name' => ['required', 'string'],
            'description' => ['nullable', 'string'],
            'sks' => ['required', 'integer', 'min:1', 'max:6'],
            'lecturer_id' => ['required', 'exists:users,id'],
        ]);

        $data['status'] = 'draft';
    }
}
```

```

        Course::create($data);

        return redirect()
            ->route('courses.index')
            ->with('success', 'Mata kuliah berhasil ditambahkan.');
```

 }

 // Menampilkan detail mata kuliah
 public function show(Course \$course)
 {
 \$course->load('lecturer');

 return view('courses.show', compact('course'));
 }

 // Menampilkan form edit
 public function edit(Course \$course)
 {
 \$lecturers = User::where('role', 'dosen')->get();

 return view('courses.edit', compact('course', 'lecturers'));
 }

 // Memperbarui mata kuliah
 public function update(Request \$request, Course \$course)
 {
 \$data = \$request->validate([
 'code' => [
 'required',
 'string',
 'unique:courses,code,' . \$course->id,
],
 'name' => ['required', 'string'],
 'description' => ['nullable', 'string'],
 'sks' => ['required', 'integer', 'min:1', 'max:6'],
 'lecturer_id' => ['required', 'exists:users,id'],
]);

 \$course->update(\$data);

 return redirect()
 ->route('courses.show', \$course)
 ->with('success', 'Mata kuliah berhasil diperbarui.');
 }

 // Menghapus mata kuliah
 public function destroy(Course \$course)
 {
 \$course->delete();

 return redirect()
 ->route('courses.index')

```
        ->with('success', 'Mata kuliah berhasil dihapus.');
```

```
    }  
}
```

4. Alur Controller

```
Request  
↓  
`routes/web.php`  
↓  
`CourseController`  
↓  
`Model Course / User`  
↓  
Database  
↓  
View  
↓  
Response
```

Checkpoint Minggu 2

1. Kenapa menghapus data lewat **GET** berbahaya? Beri satu skenario konkret.

Menghapus data menggunakan **GET** berbahaya karena **GET** seharusnya digunakan untuk mengambil data, bukan mengubah atau menghapus data.

Contoh skenario:

Jika terdapat route `/courses/5/delete` dengan method **GET**, crawler atau browser dapat mengakses URL tersebut secara otomatis. Akibatnya, mata kuliah dengan ID 5 dapat terhapus tanpa tindakan langsung dari pengguna.

2. Apa yang terjadi kalau `/courses/{course}` ditulis sebelum `/courses/create`? Kenapa?

Route `/courses/create` dapat dianggap sebagai route `/courses/{course}`, sehingga kata **create** dianggap sebagai nilai parameter `{course}`.

Akibatnya, request ke `/courses/create` dapat menjalankan method `show()` untuk mencari course dengan parameter **create**, bukan menjalankan method `create()`.

Karena itu, route statis seperti `/courses/create` harus diletakkan sebelum route wildcard `/courses/{course}`.

3. Tunjukkan di kode Anda satu tempat yang memakai `route()`. Apa untungnya dibanding URL hardcoded?

Contoh penggunaan `route()`:

```
<a href="{{ route('courses.index') }}">Mata Kuliah</a>
```